

journi

Software Requirements Specification

Human Change Management Application — Rebuild-Grade Technical Reference

Prepared for POWERACT Consulting

Version 1.0 · August 2026 · Confidential

1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
1.3 Intended Audience.....	5
1.4 Definitions, Acronyms, Abbreviations.....	6
1.5 References.....	6
1.6 Module Numbering Convention.....	6
2. Overall Description.....	7
2.1 Product Perspective.....	7
2.2 Product Functions (Summary).....	8
2.3 User Classes and Characteristics.....	8
2.4 Operating Environment.....	8
2.5 Design and Implementation Constraints.....	8
2.6 Assumptions and Dependencies.....	9
3. System Architecture.....	9
3.1 High-Level Architecture.....	9
3.2 Frontend Architecture.....	10
3.3 Backend Architecture & Persistence Model.....	10
3.4 State Synchronization.....	11
3.5 Deployment & Installation.....	12
3.6 Internationalization Architecture.....	12
3.7 Access Control Architecture.....	12
4. Data Model.....	13
4.1 Top-Level State Shape.....	13
4.2 Reference/Catalog Data (Cross-Cutting, Not Nested Per Project).....	14
4.3 Change Management Project — Field Reference.....	14
4.4 Charter Definitions (M5, Shared Catalog).....	17
4.5 Entity Relationship Summary.....	17
5. External Interface Requirements.....	18

5.1 User Interface Conventions.....	18
5.2 API Interface.....	18
5.3 Hardware Interfaces.....	19
5.4 Software Interfaces.....	19
6. Functional Requirements.....	19
6.0 Conventions.....	19
6.1 Platform & Governance (M1-M6).....	20
6.2 Change Management Program (M7-M22).....	24
7. Cross-Cutting Functional Requirements.....	33
7.1 CSV Export.....	33
7.2 Notification Center (Alerts).....	33
7.3 Justification Governance.....	34
7.4 Permission Matrix.....	34
7.5 AI Governance Tiers.....	34
7.6 Framework-to-Module Mapping.....	34
8. Non-Functional Requirements.....	35
8.1 Performance.....	35
8.2 Security.....	35
8.3 Reliability & Data Integrity.....	36
8.4 Usability & Accessibility.....	36
8.5 Localization.....	37
8.6 Portability / Installability.....	37
8.7 Maintainability / Extensibility.....	37
8.8 Compatibility.....	37
9. Seed / Reference Data Requirements.....	37
9.1 Reference Tenant.....	37
9.2 Demo User / Role Matrix.....	38
9.3 Shared Catalogs.....	38
10. Appendices.....	38

10.1 Glossary.....	38
10.2 Requirement Traceability.....	38
10.3 Known Functional Gaps (Reference Implementation).....	39
10.4 Default Permission Matrix.....	40
10.5 Closing Notes.....	41

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents *journi* — a human change-management web application — at a level of technical and functional detail sufficient for an independent development team to rebuild it from scratch: the same 22 modules, the same data model, the same access-control and localization behavior, and the same installable Windows deployment, without access to the original source code.

It is a companion to, and supersedes for rebuild purposes, the earlier *journi Functional & Module Specification*: that document describes the product's intent and framework rationale in narrative form; this one adds the missing engineering layers — data model, API surface, non-functional requirements, and per-module functional requirements with unique identifiers — and corrects two things that changed since it was written: the module numbering (all 22 modules now run in one continuous logical sequence, M1-M22, described in Section 1.6) and the persistence architecture (*journi* now ships with a real local backend, not a browser-only reference build — Section 3).

1.2 Scope

journi is a standalone, multi-tenant human change-management application. It sits alongside — but does not replace or integrate with — an organization's project-delivery/PMO tooling, and tracks the people side of a change initiative: individual and cohort readiness (ADKAR), emotional trajectory (Bridges, Kübler-Ross), organizational sponsorship and coalition strength (Kotter), and overall program state (Lewin), layered onto one shared, framework-agnostic data model.

In scope for this document: the full functional behavior of all 22 modules; the organizational hierarchy, role-based access control, and localization architecture; the data model and REST API; the Windows installation and deployment model; the governed AI Use Case Library; and the non-functional requirements (performance, security, portability, usability) the reference implementation satisfies.

Out of scope: pixel-level visual design (see the companion UI Palette artifact for the current styling direction), the illustrative seed-data narratives in full (a condensed reference appendix is included; full narratives live in the Master User Guide), and any integration with a specific third-party PMO, HRIS, or LMS system (*journi* exposes CSV export and a documented API surface as its integration points, but ships no pre-built connector).

1.3 Intended Audience

A development team tasked with rebuilding, extending, or re-platforming *journi*; a QA team writing test plans against its functional requirements; a solutions architect scoping a production (multi-tenant, cloud-hosted) deployment beyond the reference Windows installation described here.

1.4 Definitions, Acronyms, Abbreviations

Term	Meaning
ADKAR	Prosci's five-block individual change model: Awareness, Desire, Knowledge, Ability, Reinforcement
Bridges	William Bridges' Transition Model: Ending → Neutral Zone → New Beginning
Kübler-Ross	The Change Curve, adapted here for organizational (not grief) sentiment: denial, resistance, exploration, commitment
Kotter	Kotter's 8-Step organizational change process
Lewin	Unfreeze-Change-Refreeze macro-state model
RBAC	Role-Based Access Control
RACSI	Responsible / Accountable / Consulted / Sign-off / Informed — journi's extended RASCI variant
SIPOC	Suppliers-Inputs-Process-Outputs-Customers process-mapping notation
OBS	Organizational Breakdown Structure (Module M3) — the roster of named resources/roles a project draws on
WBS	Work Breakdown Structure (Module M8)
CM	Change Management (as opposed to PM, Project Management)
E2E	End-to-End (process chain, Module M4)
MP	Macro Process — an atomic unit of journi's process model (Module M4)
FR-M n - nn	Functional requirement identifier: module number, then a sequential requirement number within that module
NFR- nn	Non-functional requirement identifier

1.5 References

- *journi Functional & Module Specification* (docs-src/spec/journi_spec.md) — narrative predecessor to this document; module numbers there use the pre-renumbering internal sequence (see the translation table in Section 1.6).
- *journi — The Complete User Guide* (docs-src/master-guide) — worked walkthroughs of every module against a running scenario tenant.
- *WINDOWS_INSTALL.md* (repository root) — end-user installation instructions.
- *journi Gap Analysis* and *Gap Closure Summary* (deliverables/) — historical record of functional gaps identified and closed prior to this SRS.

1.6 Module Numbering Convention

journi's 22 modules are organized into one continuous logical sequence, M1 through M22, running from tenant creation through platform governance, the full change-management project lifecycle, and sustainment. This visible numbering is intentionally decoupled from the source file names (Module<N>Page.jsx) and route paths (/app/m<N>), which are permanent internal identifiers chosen when each module was first built and never renumbered — so a developer locating a module's code should

always use the file-path column below, not the displayed module number, to search the repository history.

Displayed #	Module Name	Source File
M1	Tenant & Org Hierarchy	Module1Page.jsx
M2	Identity & RBAC	Module2Page.jsx
M3	Organizational Breakdown Structure (OBS)	Module22Page.jsx
M4	Macro Process, SIPOC, RACSI & E2E Process Registry	Module18Page.jsx
M5	Change Management Charter Registry	Module19Page.jsx
M6	AI Use Case Library & Governance	Module16Page.jsx
M7	Initiative (CM Project) Registry	Module3Page.jsx
M8	Work Breakdown Structure & Gantt	Module17Page.jsx
M9	Stakeholder & Impact Mapping	Module4Page.jsx
M10	ADKAR Engine	Module5Page.jsx
M11	Emotional & Transition Layer (Bridges / Kübler-Ross)	Module6Page.jsx
M12	Change Risk Register	Module13Page.jsx
M13	Sponsor & Coalition	Module7Page.jsx
M14	Communication Planning & Execution	Module8Page.jsx
M15	Training & Capability Building	Module9Page.jsx
M16	Resistance Management	Module10Page.jsx
M17	Manager-as-Coach Enablement	Module11Page.jsx
M18	Journey Map / Visual Core	Module15Page.jsx
M19	Stakeholder Journeys, Touchpoints & Analytics	Module20Page.jsx
M20	Metrics & Analytics Dashboard	Module14Page.jsx
M21	Reinforcement & Sustainment	Module12Page.jsx
M22	Field Notes	Module21Page.jsx

The sidebar groups these into two sections: **Platform & Governance** (M1–M6 — tenant setup, identity, and the reference data every project draws on) and **Change Management Program** (M7–M22 — the working lifecycle of a single Change Management Project, in the order a practitioner actually touches them).

2. Overall Description

2.1 Product Perspective

journi is a self-contained, single-tenant-installable web application, not a module of a larger suite. It runs as a Node.js server process (bundled frontend + REST API) backed by a local SQLite database file, distributed today as a Windows-installable package (`install.bat` / `start-journi.bat`) but architecturally portable to any

environment able to run Node.js 18+. It has no required external service dependency — the optional exception is the AI Use Case Library's Real LLM Provider Connection (Section 6.6, M6), which is opt-in and falls back to a deterministic built-in generator when not configured.

2.2 Product Functions (Summary)

- Multi-tenant organizational hierarchy (Group → Organization → Projects) with hard data isolation between Organizations (Section 2, M1).
- Role-based access control with a runtime-editable Permission Matrix and a justification-governed audit trail for every score/state change (Section 3, M2).
- A 16-module Change Management Program lifecycle covering stakeholder mapping, ADKAR readiness, emotional/transition tracking, sponsorship, communications, training, resistance, coaching, risk, journey mapping, analytics, and sustainment (M7–M22).
- A shared, cross-cutting reference layer — Macro Processes/SIPOC/RACSI, WBS & Gantt, OBS, and a Change Management Charter Registry — that every program module draws on rather than duplicating (M3–M6, M8).
- A governed, tiered (Assistive/Augmented — never Autonomous) AI Use Case Library plugged into most modules (M6).
- Multilingual (English/French/Arabic, including full RTL) UI with tenant-default and personal-override language precedence.
- Client-side CSV export on high-traffic tables and an in-app Notification Center surfacing live-computed alert conditions.

2.3 User Classes and Characteristics

See Section 3.2 (RBAC) for the authoritative role table. In summary: platform-wide administrators (Super Admin), organization/group administrators, project-scoped practitioners (Change Manager, PMO, Trainer, Communications Practitioner), team-scoped People Managers/Coaches, read-only Sponsors and Executive Viewers, and Employees with self-service, submission-only access to their own data.

2.4 Operating Environment

- **Server:** Node.js ≥ 18 (tested through Node 24), running an Express HTTP server on a configurable port (default 4000), serving both the built frontend (static assets) and the REST API from the same origin.
- **Database:** a single local SQLite file (`server/data/journi.db`), opened via one of two interchangeable drivers — see Section 3.3.
- **Client:** any evergreen desktop browser (Chrome, Edge, Firefox); no mobile-specific layout is shipped beyond standard responsive breakpoints, though Module M17's UI is deliberately lightweight for frontline-supervisor use on a phone browser.
- **OS:** primary packaging target is Windows 10/11 (`install.bat`), but the Node/Express/SQLite stack is OS-agnostic; the same source runs on macOS/Linux with `npm install && npm start` in `server/`.

2.5 Design and Implementation Constraints

- **Frontend:** React 18 (function components + hooks only), Vite as the build tool, React Router for client-side routing, Tailwind CSS for styling, all state read from

a single React Context (AppStateContext) rather than a state-management library.

- **Backend:** Express, minimal middleware, a single persisted JSON document per tenant-scope rather than a normalized relational schema (Section 4).
- **Persistence portability:** the backend must run without requiring a native-compiled dependency to succeed at `npm install` — this is why two interchangeable SQLite drivers exist (Section 3.3); a rebuild targeting only modern Node (≥ 22 , where `node:sqlite` is stable) could drop the fallback, but the dual-driver approach is the current constraint for Windows install reliability across a wide range of installed Node versions.
- **No required cloud dependency:** the reference build must remain fully functional offline, with the sole opt-in exception of the Real LLM Provider Connection.
- **Localization-first:** every user-facing string must exist in `translations.js` for all three languages; no hardcoded UI string is acceptable outside that file.

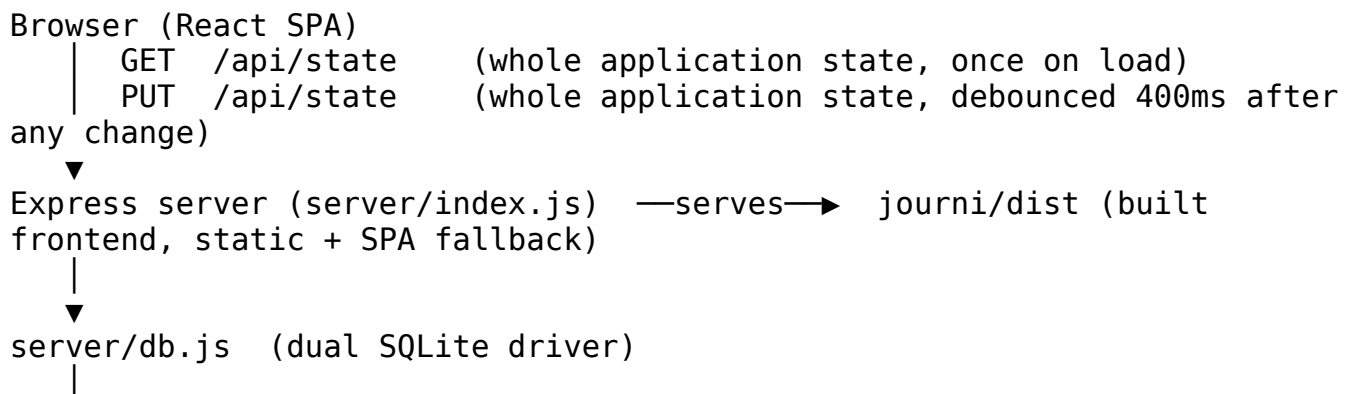
2.6 Assumptions and Dependencies

- A single journi installation serves one physical/virtual machine and one SQLite file; horizontal scaling or multi-instance deployment is out of scope for the reference build (Section 8.3 discusses what a production re-platform would need).
 - The application assumes trusted network access to the machine it runs on (e.g. a LAN or a single user's desktop) — the reference build does not implement TLS termination, rate limiting, or a WAF; a production, internet-facing deployment must add these at the infrastructure layer.
 - Demo/seed data (Section 9) is fictional and is not a substitute for real organizational data before go-live, per the Closing Notes of the predecessor Functional Specification.
-

3. System Architecture

3.1 High-Level Architecture

journi is a single Node.js process that both serves the compiled frontend and exposes the API the frontend calls, backed by one local SQLite file:





server/data/journi.db → table app_state, single row (id = 1)

There is deliberately no per-entity REST surface: the entire application's data — every organization, project, and sub-record — lives in one JSON document, and the client and server exchange that whole document rather than individual resources. Section 3.4 explains why, and what a normalized-schema rebuild would need to change.

3.2 Frontend Architecture

- **Framework:** React 18, function components and hooks only, built with Vite.
- **Routing:** React Router; each of the 22 modules is a top-level route (/app/m1 ... /app/m22), rendered inside a shared Layout (TopBar + Sidebar + routed content).
- **State:** a single React Context, AppStateContext (journi/src/state/AppStateContext.jsx), is the sole source of truth for all persisted application data (Section 4) and exposes every mutation as a function via useAppState() — no separate state-management library, no per-page local copies of server data.
- **Component library:** a small shared set used across all 22 module pages — PageHeader, Modal (the standard Add/Edit dialog shell), Badge (status/tone pill), EmptyState, ExportCsvButton, AiSuggestionBox (the standard AI-use-case card, Section 6.6), ProgressBar, JustifyPanel (the stage-then-justify governance widget, Section 6.6.5), RequireProject / RequireRole (route guards).
- **Internationalization:** journi/src/i18n/ — a single translations.js dictionary keyed by string id, each with en/fr/ar values; useI18n() resolves the active language and returns a t(key) function. Arabic renders full right-to-left, including navigation, tables and the Journey Map chart, not just mirrored text direction.
- **Data-entry pattern:** every module's Create/Edit flow follows the same shape — a modal state of { mode: 'add' | 'edit', id? } | null, a form state seeded from a BLANK_* constant (add) or the selected record (edit), and a submit() that branches into addSubItem or updateSubItem (Section 4.3) — a deliberate, repeated convention so a developer who has read one module's CRUD code has read all of them.

3.3 Backend Architecture & Persistence Model

Server: a single Express app (server/index.js) with one JSON body-parsing middleware (25MB limit, raised well above Express's 100kb default since a data-rich tenant's full state can run to several MB) and no CORS, session, or authentication middleware of any kind — see the security note in Section 8.2.

Dual SQLite driver (server/db.js): journi's own Windows-installability requirement — npm install must succeed without a native C++ toolchain on a machine that may be running anything from Node 18 to the very latest release — is met by trying two interchangeable drivers in order:

1. **node:sqlite** (Node's built-in DatabaseSync, stable from Node 22.5+) — zero npm dependency, tried first.

2. **better-sqlite3** — kept as an optionalDependency specifically so a failed native build (e.g. no Visual Studio Build Tools on a fresh Windows machine, or no prebuilt binary for a very new Node ABI) never aborts npm install as a whole; it is the fallback for Node versions below 22.5.

If neither driver loads, the server fails fast with one actionable error at import time rather than an unintelligible crash later. `driver.supportsPragma` distinguishes the two — only `better-sqlite3` receives `PRAGMA journal_mode = WAL`.

Schema — one table, one row, a JSON-blob key-value store rather than a normalized relational schema:

```
CREATE TABLE IF NOT EXISTS app_state (
  id          INTEGER PRIMARY KEY CHECK (id = 1),
  data        TEXT,          -- JSON.stringify of the entire frontend
  `data` object (Section 4)
  current_user_id TEXT,
  scope       TEXT,          -- JSON.stringify of { orgId, cmProjectId
}
  updated_at  TEXT NOT NULL
)
```

This is a deliberate design choice, not an oversight: the frontend’s own evolving state shape (`migrateOrSeed()` in `AppStateContext.jsx`) is treated as the single source of truth for what the data looks like, so a normalized SQL schema would be a second, driftable copy of the same shape. A production re-platform that needs per-entity querying, reporting joins, or multi-writer concurrency should normalize this into real tables (Section 8.3 discusses the tradeoff explicitly) — but should preserve the JSON field shapes in Section 4 as the schema-design input, since every module’s frontend code is written directly against them.

Storage location: `${JOURNI_DATA_DIR:-<server>/data}/journi.db`, directory auto-created if missing.

3.4 State Synchronization

Persistence is debounced client-side, not per-mutation, and every write is a full-document overwrite:

1. On app load, the client fetches `GET /api/state` once. If the backend already holds data, it replaces the initial in-memory demo seed (rendered immediately for a no-flash first paint) with the server’s data, `currentUserId`, and `scope`.
2. Every subsequent state change re-arms a 400ms debounce timer; when it elapses, the client sends the **entire** `{ data, currentUserId, scope }` object as the body of `PUT /api/state`, coalescing bursts of rapid edits (e.g. typing in a field) into one write.
3. The server does not merge or patch — it JSON-stringifies the body and upserts the single `app_state` row. There is no optimistic-locking or versioning at the transport level: the last write to finish wins.
4. A failed save is swallowed silently and simply retried on the next state change; nothing is queued client-side beyond ordinary React state.

A developer tracing “why did my edit not save”: check that the 400ms debounce elapsed, that `loadedFromServer` was already true when the edit happened (edits made before the initial GET `/api/state` resolves are held in memory and included in the next debounced save once it does), and that the server process could reach its SQLite file.

3.5 Deployment & Installation

The distributed package (`deliverables/journi_fullstack_windows.zip`) ships source only — `journi/` (frontend source, no `node_modules` or `dist`) and `server/` (backend source, no `node_modules`) — plus `install.bat`, `start-journi.bat`, and `WINDOWS_INSTALL.md`. `install.bat` runs `npm install` in both `journi/` and `server/`, then `npm run build` in `journi/` to produce `journi/dist`, which `server/index.js` serves statically with an SPA fallback (any unmatched path returns `index.html` so client-side routing works on a hard refresh/deep link). `start-journi.bat` launches `node server/index.js`, which listens on `process.env.PORT || 4000` and logs the resolved SQLite file path on startup. A rebuild targeting a different OS needs only `npm install && npm run build` (in `journi/`) and `npm start` (in `server/`) — the batch files are a Windows-convenience wrapper around otherwise standard Node tooling.

3.6 Internationalization Architecture

Every user-facing string is keyed in `journi/src/i18n/translations.js` with `en/fr/ar` values — no hardcoded UI string is acceptable outside that file (Section 2.5). Language precedence, resolved on every sign-in and every Organization switch: (1) the signed-in user’s own explicit language preference, if set; (2) the newly-scoped Organization’s configured Default Language (set on M1 by a Super Admin, Group Admin, or Organization Admin); (3) English as the platform fallback. A manual in-session language switch applies immediately but is session-local only — it never overwrites the stored user preference or the Organization default, and is re-resolved from the precedence order above on the next sign-in or Organization switch. Arabic renders full right-to-left: navigation, tables, and the Journey Map’s chart mirror correctly, not just text direction.

3.7 Access Control Architecture

See Section 6.6.1–6.6.3 (M2) for the full Permission Matrix and Justification Governance functional requirements; this section summarizes the architecture. RBAC is enforced **entirely client-side** (`journi/src/utils/rbac.js`) — the API itself performs no authorization check of any kind (Section 5.2, Section 8.2). Nine roles (`super_admin`, `group_admin`, `org_admin`, `sponsor`, `change_manager`, `people_manager`, `practitioner`, `employee`, `executive`) are each scoped to one level of the Group → Organization → Project hierarchy. Each write/manage capability check (`canWrite`, `canManageHierarchy`, `canManageUsers`, `canActivateAiForOrg`, `canManageCharters`, `canManageAiUseCases`, `canManageTemplates`, `canRequestProjectAiOverride`) reads first from the runtime-editable Permission Matrix (`data.rolePermissions`, a role × capability grid seeded with sensible defaults and editable in place by a Super Admin on M2), falling back to hardcoded default logic only where a matrix entry is missing — so the matrix is a fail-safe override, never a bypass of the underlying model.

4. Data Model

journi persists one JSON document per installation (Section 3.3). This section is the authoritative field-level schema a rebuild must reproduce; every field name below is drawn directly from the reference implementation's default-object constants and seed data, not inferred.

4.1 Top-Level State Shape

Key	Holds
groups	Array of Group objects — optional top tier of the hierarchy (a holding company / multi-entity corporate group).
organizations	Array of Organization objects — the primary tenant boundary.
mainProjects	Array of Main Project objects — the underlying business/IT initiative (e.g. an ERP rollout), distinct from a Change Management Project.
cmProjects	Array of Change Management Project objects — the core, heavily-nested working entity (Section 4.3).
users	Array of platform user/account objects (identity + RBAC scope).
aiUseCaseCatalog	Array of AI Use Case definitions — the shared, versioned catalog (M6).
macroProcessCatalog	Array of Macro Process reference entries (M4, MP-01...MP-10).
e2eProcessCatalog	Array of End-to-End Process Registry entries (M4, E2E-xx).
phaseTemplateCatalog	Array of Phase Template definitions — shared, versioned (M8).
racsiGrid	Object keyed by Macro Process id → { [role]: 'R' 'A' 'C' 'S' 'I' } (M4 RACSI grid, editable).
aiOrgActivation	Object keyed by orgId → { [useCaseId]: boolean } — whether an AI use case is switched on for an Organization (M6).
aiProjectOverride	Object keyed by cmProjectId → { [useCaseId]: boolean } — per-project override of the Organization default (M6).
aiUsageLog	Array of AI usage/outcome audit-log entries, platform-wide (not nested per project).
codebooks	Object keyed by orgId → array of qualitative-coding Code objects (M16 Coding Workbench, org-scoped taxonomy).
charters	Array of CM Charter definitions (M5), each with a nested entries[] (Section 4.4).
rolePermissions	Object keyed by role → { [capabilityKey]: boolean } — the runtime-editable Permission Matrix (M2).
requireJustification	Boolean — platform-wide Justification Governance flag (M2).

Key	Holds
license	Single License object — SaaS-mode demo license record (M2).

currentUserId and scope ({ orgId, cmProjectId }) are persisted alongside data (same server row, sibling fields) but are not part of data itself.

Not persisted server-side by design: the Real LLM Provider Connection's configuration ({ provider, apiKey, model, baseUrl }) lives only in the browser's localStorage, under a separate key from the rest of the application's data, so an API key is never bundled into a data export, never synced across machines, and survives a "Reset Demo Data" action untouched.

4.2 Reference/Catalog Data (Cross-Cutting, Not Nested Per Project)

These arrays back M3–M6 and M8 and are shared across every Organization and Project rather than duplicated per tenant: aiUseCaseCatalog, macroProcessCatalog, e2eProcessCatalog, phaseTemplateCatalog, racsiGrid, charters (definitions; compliance logging is per-project, Section 4.3), plus the constant reference tables described per-module in Section 6 (ADKAR block list, resistance types, RESISTANCE_TYPES, journeys/journeyTouchpoints, macroProcesses, e2eProcesses, crossTypeMatrix, benchmarks, mentoringStages, defaultCodebook, alertDefinitions — each a static JS module under journi/src/data/, not a database table, since they change only with a product release, not per-tenant).

4.3 Change Management Project — Field Reference

A cmProject object's own top-level fields: id, orgId, mainProjectIds[] (many-to-many link to zero, one, or more Main Projects — Section 6.1, M1), name, changeManager, changeType, businessDriver, targetPopulation, successCriteria, lewinPhase, bridgesPhase, bridgesNote, sentimentSnapshot, aiUseCases[] (activated-use-case-id strings), changeLog[], adkar{}, sponsor{}, sustainment{}, plus every sub-collection array below.

Collection	Module	Item shape
adkar (object, keyed by block)	M10	{ score: 1-5, note, history: [{ id, date, score, justification? }] }
changeLog	M2 (governance)	{ id, date, module, field, oldValue, newValue, justification } — the Justification Governance audit trail (Section 6.6.2)

Collection	Module	Item shape
risks	M12	{ id, category: 'adoption'\\'sponsorship'\\'capacity'\\'saturation', description, likelihood: 1-5, impact: 1-5, owner, status: 'open'\\'mitigating'\\'closed', actions?: [{ id, description, owner, dueDate, status: 'open'\\'in_progress'\\'closed' }] } — actions[] is the Mitigation Action Plan
stakeholderGroups	M9	{ id, name, headcount, impact: { process, tech, role, location, identity } (each 1-5), influence: 1-5 }
communications	M14	{ id, message, audience, channel, sender, timing, adkarBlock, status: 'draft'\\'scheduled'\\'sent' }
trainings	M15	{ id, curriculum, track, level: 'Foundation'\\'Practitioner'\\'Advanced', module, objectives, prerequisites, agenda, targetAudience, expectedResults, workshops, facilitator, format, completion: 0-100, certified: boolean }
resistanceLog	M16	{ id, type: 'role'\\'skill'\\'will'\\'systemic', source, rootCause, severity: 1-5, mitigation, owner, dueDate, status: 'open'\\'in_progress'\\'closed', anonymous: boolean }
coachingNotes	M16 / M17	{ id, managerName, cohort, barrierBlock, note }
journeyEvents	M18	{ id, offsetDays, label, type: 'milestone'\\'communication'\\'assessment'\\'training' }

Collection	Module	Item shape
wbsTasks	M8	{ id, track: 'pm'\ 'cm'\ 'framework', phase, name, baselineStart, baselineEnd, actualStart, actualEnd, accountabilityTag: 'PROJECT'\ 'CHANGE'\ 'JOINT' , status: 'planned'\ 'in_progress'\ ' done'\ 'at_risk', percentComplete: 0-100 }
phaseGates	M8	{ id, phase, date, jointDecision: 'go'\ 'go_with_conditions'\ 'no_go', conditions, accountable: 'PM'\ 'CM'\ 'ES'\ 'FP0'\ 'I' TL'\ 'SUP', pmInput: { recommendation, notes }, cmInput: { recommendation, notes, readinessIndexSnapshot, checklistCompletionPct, openFlags } }
phaseChecklists	M8	{ id, phase, track: 'pm'\ 'cm', item, weight: 1-100, done: boolean }
charterActionLog	M5	{ id, charterActionId (FK → static charter-actions catalog), date, note }
touchpointLog	M19	{ id, touchpointId (FK → static journeyTouchpoints catalog), date, note }
codeTags	M16	{ id, codeId (FK → codebooks[orgId]), sourceType, sourceId, linkedResistanceId: FK → resistanceLog \ null, date }
dismissedAlerts	Notification Center	array of alert-id strings — the alerts themselves are always computed live (Section 6.6.4), never persisted; only dismissal state is
fieldNotes	M22	{ id, date, category, relatedModules: string[] (multi-select), title, body, author }
obsEntries	M3	{ id, role, name, reportsTo: FK → another entry in the same project's obsEntries \ null, notes }

Collection	Module	Item shape
sponsor (object)	M13	{ name, visibility: 'weak'\\'moderate'\\'strong', visibilityNote, members: [{ id, name, role, influence: 1-5, engagement: 1-5 }], actions: [{ id, action, phase: 'Prepare'\\'Manage'\\'Reinforce', done: boolean }] }
sustainment (object)	M21	{ checkpoints: [{ id, label: '30-day'\\'60-day'\\'90-day', daysAfterGoLive, adoptionRate, regressionRisk: 'low'\\'moderate'\\'high'\\null, status: 'not_due'\\'complete' }], quickWins: [{ id, title, date }], lessonsLearned: [{ id, text, linkedRuleOrControl?, status?: 'pending'\\'applied' }], signoff: boolean }

Computed-only values (never stored — recomputed from the fields above on every render): riskScore/isHighSeverityRisk (M12), taskGapDays/gapTone (M8), phase-checklist completion percentage (M8), the Composite Readiness Index and divergence flags (M20), and every live alert condition (Notification Center).

4.4 Charter Definitions (M5, Shared Catalog)

charters[] (top-level, not per-project): { id, name, category, what, who, when, where, why, how, ownerRole, racsi, governingScope, version, reviewCadence, status: 'Active'\\'Retired', entries: [{ ...action-mapping fields – pdcaStage, sequence, linkedMacroProcess/Task/Phase, responsibleRole, accountableRole, frequency }] }.

4.5 Entity Relationship Summary

Group (0..1) —< Organization >— License (per-org context via M2)
Organization —< MainProject
Organization —< CmProject
MainProject >—< CmProject (many-to-many via cmProject.mainProjectIds[])
Organization —< User (scoped by scopeType/scopeId to org, group, or a specific project)
CmProject —< every sub-collection in Section 4.3 (1-to-many, embedded arrays – not separate tables)
Organization —< aiOrgActivation entry
CmProject —< aiProjectOverride entry
Organization —< codebooks entry —< codeTags (per CmProject, referencing codebooks[orgId])

No sub-collection item is ever referenced from more than one `cmProject` — every relationship below the `CmProject` level is a plain embedded array, not a foreign-keyed table, which is what makes the single-JSON-document persistence model in Section 3.3 a faithful fit rather than a shortcut.

5. External Interface Requirements

5.1 User Interface Conventions

- **Shell:** a fixed TopBar (wordmark, Organization/Project scope selector, Notification Center bell, current user) and a Sidebar (Section 1.6’s two-section module list), with routed content filling the remaining viewport.
- **CRUD pattern:** every module’s Add/Edit flow opens the shared Modal component; every list view pairs an Edit (secondary-style) and Delete (danger-style) button per row, gated by the same role/capability check as the module’s own write access (Section 3.7).
- **Status encoding:** the shared Badge component renders a colored pill (tone: brand/green/amber/red/gray/sand) for every status, severity, or classification field across all 22 modules — journi’s shared visual vocabulary for “attention needed.”
- **RTL:** every layout listed above mirrors correctly under Arabic (Section 3.6), not just inline text.

5.2 API Interface

The backend exposes exactly two functional endpoints — this is a whole-state store, not a per-resource REST API; all entity-level create/update/delete logic lives client-side (Section 3.2, 4.3) and is expressed on the wire only as the net effect on the one JSON document below.

Method	Path	Request Body	Response	Notes
GET	/api/health	—	{ ok: true }	Liveness check.
GET	/api/state	—	{ data, currentUserId, scope }, or { data: null, currentUserId: null, scope: { orgId: null, cmProjectId: null } } on an empty database; 500 { error } on failure	Reads the entire application state. No authentication, no scoping/filtering by caller — returns everything.
PUT	/api/state	{ data, currentUserId, scope } (the client’s entire in-memory state)	{ ok: true }; 500 { error } on failure	Full blind overwrite of the single stored row — no merge, no shape validation, no optimistic-locking/versioning.

Method	Path	Request Body	Response	Notes
GET	*	—	Static file from <code>journi/dist</code> if present, else <code>index.html</code> (SPA fallback)	Frontend hosting; if <code>journi/dist</code> is missing, returns 500 with an actionable build-reminder message instead.

A production rebuild that needs per-entity access control, partial updates, audit logging at the transport layer, or multi-writer concurrency should replace this with a conventional resource-oriented API (one route family per entity in Section 4) — the field shapes in Section 4.3 are the contract to preserve when doing so, since every module’s frontend code already expects them.

5.3 Hardware Interfaces

None. `journi` is a standard web application with no direct hardware interface requirement.

5.4 Software Interfaces

- **Node.js** ≥ 18 runtime (Section 2.4).
- **SQLite**, via `node:sqlite` or `better-sqlite3` (Section 3.3) — no external database server.
- **Optional**: a third-party LLM provider (Anthropic, OpenAI, Google, or a custom OpenAI-compatible endpoint) for M6’s Real LLM Provider Connection, called directly from the browser — deliberately bypassing `journi`’s own backend even though one now exists for state persistence, so that a production, multi-user deployment is not forced to store third-party API keys server-side; the connection is opt-in and every AI use case falls back to a deterministic built-in generator if it is not configured or a call fails.

6. Functional Requirements

6.0 Conventions

Each module below lists: its purpose and the framework(s) it operationalizes; the fields it captures; numbered functional requirements (FR-Mn-*nn*); the business rules/computations it applies; its role gating; and its cross-module integration points. Every module follows the same shared CRUD, RBAC, and AI-governance conventions described in Sections 3.2, 3.7, and 6.6 — those are not repeated per module except where a module’s actual behavior deviates from them, which is called out explicitly rather than silently assumed. Section 10.3 (Known Functional Gaps) collects every such deviation in one place for planning purposes.

6.1 Platform & Governance (M1-M6)

6.1.1 M1 — Tenant & Org Hierarchy

Purpose. The structural backbone of journi: Group (optional) → Organization → Projects, and within Projects, a Main Project (the underlying business/IT initiative) many-to-many linked to zero or more Change Management Projects (the people-side initiative that manages its adoption).

Data captured. Group: name. Organization: name, groupId, sector, employeeCount, sites, languages, defaultLanguage. Main Project: name, type (one of ten values — the eight transformation types plus Restructuring and M&A), scope, durationMonths, budgetBand, executiveSponsor. Change Management Project: name, orgId, mainProjectIds[], changeManager, changeType, targetPopulation, businessDriver, successCriteria.

Functional requirements. - FR-M1-01: The system shall support full Create, Read, Update, and Delete on Group, Organization, Main Project, and Change Management Project records, gated by canManageHierarchy (Section 3.7). - FR-M1-02: A Change Management Project shall link to zero, one, or more Main Projects via mainProjectIds[] — a many-to-many relationship, not the one-to-one link of earlier product iterations — supporting both a fully independent OCM initiative (no Main Project link) and a multi-site rollout where several regional CM Projects share one Main Project. - FR-M1-03: Deleting a Group shall not delete its member Organizations; it shall null their groupId, converting them to standalone Organizations. - FR-M1-04: Deleting an Organization shall cascade-delete its Main Projects, Change Management Projects, associated AI activation/override state, and any user accounts scoped directly inside it. - FR-M1-05: Deleting a Main Project shall unlink it from (not delete) any Change Management Project that referenced it via mainProjectIds[]. - FR-M1-06: Deleting a Change Management Project shall also remove any user accounts scoped directly to that project. - FR-M1-07: The Organization record shall carry a Default Language field, editable by the same role set, driving the language-precedence rule in Section 3.6. - FR-M1-08: The hierarchy view shall display, for each Main Project, the count of Change Management Projects currently linked to it, and shall badge a Change Management Project with no Main Project link as “standalone.”

Frameworks integrated. None directly — this is the structural layer every framework-bearing module (M7 onward) is scoped against.

Primary users. Super Admin, Group Admin, Organization Admin, PMO.

6.1.2 M2 — Identity & RBAC

Purpose. User account management, the runtime-editable Permission Matrix, platform-wide governance toggles, and a lightweight license/plan record. Organized into four tabs: Users & Scope, Permission Matrix, Governance Settings, License & Plan.

Data captured. User: name, email, role (one of the nine roles in Section 3.7), scopeType, scopeId, language. Permission Matrix: 9 roles × 8 capabilities (manageHierarchy, manageUsers, write, activateAiForOrg, requestProjectAiOverride, manageCharters, manageAiUseCases,

manageTemplates). Governance: requireJustification (platform-wide boolean). License: mode (SaaS/OnPrem), plan, companyName, maxUsers, issueDate, expiryDate, features[], and (OnPrem) an uploaded .lic JSON file's companyId/hardwareId.

Functional requirements. - FR-M2-01: The system shall support Create, Update (role only), and Delete on User records, gated by canManageUsers. Name, email, and scope are set at creation and are not independently editable after — a rebuild targeting fuller account-lifecycle management should add this. - FR-M2-02: The Permission Matrix tab shall render all 9 roles × 8 capabilities as an editable grid, editing gated to super_admin only regardless of any Permission Matrix entry (this one gate is intentionally hardcoded, not itself matrix-driven, to prevent a misconfiguration from locking every admin out of the matrix). - FR-M2-03: Every edit to a Permission Matrix cell shall take effect immediately, platform-wide, for every Organization, without a code change or restart. - FR-M2-04: The Governance Settings tab shall expose a single requireJustification toggle, defaulting to true, gated by canManageHierarchy. - FR-M2-05: The License & Plan tab shall display current plan, seat usage against maxUsers with an over-capacity flag when exceeded, and an expiry countdown color-coded red (expired), amber (<30 days), or green (otherwise). - FR-M2-06: Uploading a .lic file shall validate the presence of required fields (version, companyId, companyName, hardwareId, expiryDate, maxUsers, plan, features, issueDate) but does not verify a cryptographic signature — this is a reference-build simplification, not a production licensing control; a Super Admin may revert to SaaS mode at any time.

Frameworks integrated. None directly — this module is the control plane every other module's write-gating reads from.

Primary users. Super Admin, Group/Organization Admin, all downstream roles (read of their own account).

6.1.3 M3 — Organizational Breakdown Structure (OBS)

Purpose. A per-project resourcing roster — role, named person, and reporting line — distinct from M2's login/RBAC user accounts: an OBS entry needs no journi login, since it exists to be *referenced* (as an assignee) by other modules, not to authenticate.

Data captured. role (free text), name, reportsTo (id of another OBS entry in the same project, or null for the top of the structure), notes.

Functional requirements. - FR-M3-01: The system shall support full Create, Read, Update, and Delete on OBS entries, gated by canWrite. - FR-M3-02: Deleting an OBS entry shall re-parent its direct reports to the deleted entry's own manager (reportsTo) rather than leaving them orphaned. - FR-M3-03: The roster view shall render depth-first, roots first, with indentation reflecting reporting depth, not creation order. - FR-M3-04: The OBS roster shall be consumable as an "Assigned to" source by other modules (M8's WBS tasks, at minimum), so a task can name a specific person from this project's own resourcing structure rather than free text.

Frameworks integrated. None directly — supporting reference data for M8.

Primary users. Change Manager, PMO, Practitioner, People Manager.

6.1.4 M4 — Macro Process, SIPOC, RACSI & End-to-End Process Registry

Purpose. The process backbone every other module is built on, made browsable in one place. Ten Macro Processes (MP-01–MP-10) are the atomic units of journi’s process model; every module owns one or more of them. Seventeen End-to-End (E2E) Process chains are registered against that backbone, grouped by kind: core lifecycle chains spanning the whole engagement, cross-cutting loop chains making an existing cross-module dependency explicit as its own registered chain (e.g. the PM ↔ CM Governance Bridge realized by M8’s Phase Gates), and one E2E lifecycle per transformation type, each with its own rolled-up SIPOC and linked Phase Template (M8).

Data captured (reference, mostly read-only). Macro Process: id, name, description, primary modules. E2E Process: id, name, kind, ordered Macro Process chain, trigger, terminal state, RACSI (using the 7-code role taxonomy ES/CM/PM/FPO/ITL/SUP/EU), related modules, and — for transformation-type entries — SIPOC suppliers/customers and a linked Phase Template id. RACSI Grid: one row per Macro Process × 6 platform roles.

Functional requirements. - FR-M4-01: The Macro Process Catalog and E2E Process Registry shall be read-accessible to every role; this reference data is not user-editable in the current build (a future release adding admin-editable process taxonomy should preserve read access for all roles as a baseline). - FR-M4-02: The RACSI Grid tab shall be editable in place, one cell at a time, gated by canManageHierarchy; every other role sees it read-only. - FR-M4-03: An E2E Process entry of kind “transformation type” shall resolve and display its linked Phase Template by id, and its rolled-up SIPOC.

Frameworks integrated. Cross-cutting — the canonical source for the Macro Process and E2E Process vocabulary referenced throughout this document; also hosts the RACSI governance layer referenced by M8’s Phase Gates.

Primary users. All roles (read); Super Admin, Group/Organization Admin (RACSI grid edits).

6.1.5 M5 — Change Management Charter Registry

Purpose. Eight signed, trackable behavioral standards — one each for sponsorship, participative management, communication, organizational impact, team coaching, one-to-one coaching, mentoring, and pulse/interview discipline — each naming specific, observable actions with a concrete mapping to the Macro Processes/tasks that generate them, plus a per-project compliance log. Organized into three tabs: Charters, Charter Action Mapping, and a read-only Mentoring Progression reference (the three-stage Trainee → Observer → Autonomous competency model).

Data captured. Charter: name, PDCA cycle, RACSI, primary linked Macro Process, governing scope (Project/Organization/Group), status (Active/Draft/Retired), version, effective date, review frequency, and one or more Entries (category, owner, What/Who/When/Where/Why/How). Charter Action (static reference): PDCA stage, sequence, category, linked Macro Process/task/step/lifecycle phase, responsible/accountable role, frequency. Charter Action Log (per project): charterActionId, date, note.

Functional requirements. - FR-M5-01: The system shall support full Create, Read, Update, and Delete on Charters and their Entries, gated by `canManageCharters`. - FR-M5-02: A Charter with status “Active” shall not be deletable; it must be transitioned to “Retired” before its Delete control is enabled. Delete itself is gated separately by `canDeleteCharter`, a fixed set (Super/Group/Org Admin) that deliberately excludes Change Manager even though Change Managers can create and edit charters. - FR-M5-03: The Charter Action catalog and Mentoring Progression model are static reference data with no CRUD UI. - FR-M5-04: The system shall support Create and Delete (not Update) on Charter Action Log entries, gated by `canWrite`; correcting a logged entry is done by delete-and-relog rather than in-place edit. - FR-M5-05: The Action Mapping & Compliance tab shall be filterable by charter and shall show, per action, up to three recent log instances plus a count of any remainder.

Frameworks integrated. Prosci Sponsor Model and Kotter Step 2 (Sponsorship Charter); Kotter Step 5 (Participative Management); Kotter Step 4 (Communication); Prosci Impact & Stakeholder Analysis (Organizational Impact); the Divergence Case recovery workflow (One-to-One Coaching); M15’s trained-versus-capable distinction (Mentoring).

Primary users. Change Manager, Training Lead, People Manager (compliance logging); all roles (read).

6.1.6 M6 — AI Use Case Library & Governance

Purpose. The governed catalog and control plane for every AI capability in *journi*. Every use case is restricted to one of two tiers — Assistive or Augmented — deliberately excluding Autonomous AI: no use case may take an irreversible action on its own. Also hosts the optional Real LLM Provider Connection every `AiSuggestionBox` instance across the program modules calls into.

AI tiers. *Assistive*: the AI observes, analyzes, or suggests; a human performs the task and decision (e.g. flagging a barrier pattern). *Augmented*: the AI performs a substantial part of the task — drafting, classifying, summarizing at scale — but a human must review, edit, and approve before it is finalized or distributed (e.g. drafting an executive readiness narrative). Autonomous AI is out of scope entirely; a future release adding it must govern it as a separate, explicitly-gated tier rather than folding it into these two.

Data captured. AI Use Case (14 seeded, versioned): id, name, tier, module, description, trigger, output, an editable `promptTemplate` (the literal text sent to the LLM), `humanCheckpoint`, version history. LLM Provider Connection (browser-local only — Section 4.1): provider, `apiKey`, model, `baseUrl`. AI Usage Log: `useCaseId`, `orgId`, `cmProjectId`, `outputSummary`, outcome (accepted/edited/rejected), user, timestamp.

Functional requirements. - FR-M6-01: The system shall support full Create, Read, Update, Delete, and version-revert on the AI Use Case catalog, gated by `canManageAiUseCases`. - FR-M6-02: Deleting a use case shall also purge its keys from the organization and project activation maps. - FR-M6-03: An Organization Admin or above (`canActivateAiForOrg`) shall be able to activate or deactivate each use case independently per Organization. - FR-M6-04: A Change Manager or above (`canRequestProjectAiOverride`) shall be able to override the Organization’s activation state for one specific project, as a tri-state choice (inherit / on / off); effective

activation is the project override if set, else the Organization’s activation. - FR-M6-05: Deactivating a use case shall immediately stop new AI-generated suggestions in that scope; previously generated and human-approved content shall never be retroactively removed. - FR-M6-06: Every AI-generated output shall be visually labeled as AI-generated and requiring review, wherever it appears in the interface. - FR-M6-07: Every AI suggestion’s outcome — accepted as-is, edited, or rejected — shall be logged to the AI Usage Log for governance reporting; the log itself is append-only, with no edit or delete UI. - FR-M6-08: The Real LLM Provider Connection shall store its configuration, including the API key, only in the browser’s `localStorage`, under a key namespace separate from the rest of the application’s data, so it is never included in a data export, never synced to the backend, and survives a “Reset Demo Data” action untouched (Section 4.1, Section 5.4). - FR-M6-09: If a configured LLM call fails for any reason (missing key, network/CORS error, unexpected response), the calling use case shall fall back to its deterministic built-in generator automatically, so a misconfigured or absent connection never blocks a workflow. - FR-M6-10: An AI use case shall never grant a user visibility they would not otherwise have under RBAC — it operates strictly within the data-visibility boundary of the module it plugs into.

Frameworks integrated. Cross-cutting — accelerates the analysis/drafting work within every framework-bearing module; tier discipline follows POWERACT’s own AI-training-curriculum vocabulary.

Primary users. Super Admin, Group/Organization Admin (activation, provider connection, catalog authoring); Change Manager, Trainer, Communications Practitioner, People Manager (consumption, all program modules).

6.2 Change Management Program (M7-M22)

6.2.1 M7 — Initiative Registry

Purpose. The single-project detail view and organization-wide portfolio table for a Change Management Project: core metadata, the Lewin macro-state (Unfreeze/Change/Refreeze) under justified-change governance, the live Composite Readiness Index, and the full justified-change audit log (the “Change Log”).

Data captured. `changeType`, `lewinPhase` (justified), `businessDriver`, `targetPopulation`, `successCriteria`; linked Main Project(s), read-only here (edited on M1); Change Log entries (date, module, field, `oldValue`, `newValue`, justification).

Functional requirements. - FR-M7-01: `businessDriver`, `targetPopulation`, `successCriteria`, and `changeType` shall be editable in place (auto-saved on blur) via `updateProjectMeta`, gated by `canWrite`. - FR-M7-02: A change to `lewinPhase` shall follow the stage-then-justify pattern (Section 6.6.2): a pending value plus a justification note, saved atomically to the Change Log, with Save disabled until a justification is present whenever `requireJustification` is on. - FR-M7-03: The Change Log shall be append-only — no edit or delete of a logged entry — and shall aggregate justified changes logged from this module and from every other module that calls the shared `logJustifiedChange/updateAdkar` mutators (M10, M11, M13, M15, M16, M17, M12). - FR-M7-04: The system shall compute a Composite Readiness Index per project as $(\text{ADKAR average \%} \times 0.5) + (\text{sentiment score} \times 0.25) + (\text{training completion average \%} \times 0.25)$, displayed both on the single-project view and in the organization-wide portfolio table. - FR-M7-05: The portfolio table shall list every Change

Management Project visible to the signed-in role within the current Organization, with its Lewin phase and Readiness Index.

Frameworks integrated. Lewin (Unfreeze-Change-Refreeze) as the organizing macro-state; Prosci’s 3-Phase Process (Prepare, Manage, Reinforce) as the default project lifecycle.

Primary users. PMO, Change Manager, Sponsor.

6.2.2 M8 — Work Breakdown Structure & Gantt

Purpose. A single Work Breakdown Structure spanning three tracks — Project Management, Change Management, and the Lewin/Prosci/Bridges/ADKAR framework milestones — so a Change Manager, PMO, and Sponsor share one timeline. Every task carries a baseline and an optional actual date pair, surfacing a schedule gap task-by-task and as a portfolio average. Also hosts Phase Checklists, Phase Gate joint-decision records, a read-only cross-reference into M5’s Charter Actions log, and the shared Phase Template library.

Data captured. WBS Task: track (pm/cm/framework), accountability tag (Project/Change/Joint — deliberately distinct from track), phase, name, assignee (from M3’s OBS roster), baseline start/finish, optional actual start/finish, status. Phase Checklist item: phase, track, item, weight%, done. Phase Gate (“Joint Decision Record”): phase, date, independent PM and CM recommendations with notes (the CM input auto-carrying a Composite Readiness Index snapshot, checklist completion %, and open flags), the fused joint decision (Go / Go with Conditions / No-Go), and exactly one Accountable role selected from the RACSI role-code list. Phase Template (shared, versioned, 8 seeded): per-phase name, CM-track actions, checklist items, gate review questions.

Functional requirements. - FR-M8-01: The system shall support full Create, Read, Update, and Delete on WBS tasks. - FR-M8-02: The system shall support Create, Read, and Delete (done-toggle counts as the only field-level update) on Phase Checklist items. - FR-M8-03: The system shall support Create and Delete on Phase Gates; a recorded Joint Decision is treated as immutable once logged and has no Edit — correcting one requires deleting and re-recording it. A rebuild wanting mutable gate records should add this deliberately, not assume it already exists. - FR-M8-04: “Load Phase Template” shall seed the PM track with one skeleton task per phase of a selected Phase Template, evenly spaced from a chosen start date, defaulting to the template matching the linked Main Project’s transformation type; this is a starting skeleton, not a finished plan, and does not itself constitute editing the template. - FR-M8-05: The Phase Template library shall support full Create, Update, Delete, and version-revert, gated by `canManageTemplates`, independent of the per-project WBS CRUD above. - FR-M8-06: The system shall compute, per task with an actual date recorded, a schedule-gap value in days, and shall render it color-coded (on-time/ahead, minor slip, significant slip); a portfolio-level summary shall show total tasks, tasks on-track or ahead, tasks at risk, and the average gap. - FR-M8-07: A framework-track task may be zero-duration (baseline start equals finish), rendered as a marker rather than a bar, for discrete state transitions (e.g. “Unfreeze → Change”) rather than continuous work. - FR-M8-08: Every task, checklist item, and gate shall additionally be filterable against the seven generic P1-P7 lifecycle phases that every type-specific Phase Template rolls up into, auto-mapped from the task’s own phase label. - FR-M8-09: A

Phase Gate's CM input fields (readiness index snapshot, checklist completion %, open flags) shall auto-populate from live computed values (M7's Readiness Index, this module's own checklist completion, and M11/M16's divergence/stalled-block flags) at the moment the gate modal opens. - **Known deviation** (Section 10.3): this module's write-access check calls `canWrite(currentUser?.role)` without passing the Permission Matrix, unlike every other module — so a Super Admin's runtime edits to the Permission Matrix do not affect M8's write gating, which always falls back to the hardcoded default role set. A rebuild should pass the matrix here for consistency with the rest of the application, unless this is confirmed as an intentional exception.

Frameworks integrated. Cross-cutting — gives Lewin, Prosci, Bridges, and ADKAR a shared timeline alongside the Project Management delivery track; Phase Gates implement the PM ↔ CM Governance Bridge cross-cutting E2E chain (M4).

Primary users. PMO, Change Manager, Sponsor.

6.2.3 M9 — Stakeholder & Impact Mapping

Purpose. Identifies who is affected by a change, how heavily, and in what dimension, so effort (deep ADKAR tracking versus light-touch communication) is proportional to risk.

Data captured. name, headcount, impact across five dimensions (process, technology, role, location, identity, each 1-5), influence (1-5).

Functional requirements. - FR-M9-01: The system shall support full Create, Read, Update, and Delete on stakeholder groups, gated by `canWrite`. - FR-M9-02: The system shall automatically flag a stakeholder group as "High Impact / Low Influence" when its average impact across the five dimensions is ≥ 3.6 and its influence is ≤ 2 — the population most at risk of being under-supported. - FR-M9-03: Each impact-dimension cell shall be color-coded by severity (red ≥ 4 , amber = 3, brand-neutral < 3).

Frameworks integrated. Prosci Impact & Stakeholder Analysis; determines which cohorts receive deep ADKAR tracking on M10.

Primary users. Change Manager, People Managers, PMO.

6.2.4 M10 — ADKAR Engine

Purpose. The heart of person/cohort-level tracking: the five ADKAR building blocks (Awareness, Desire, Knowledge, Ability, Reinforcement), each under justified-change governance, plus an AI-assisted Desire-barrier diagnosis-and-coaching workflow and manager coaching-note capture.

Data captured. Per block: score (1-5), justification note, history (date, score, justification). Coaching Note: manager name, cohort, barrier block, note.

Functional requirements. - FR-M10-01: A change to any ADKAR block's score shall follow the stage-then-justify pattern, appended to that block's own history and to the project Change Log. - FR-M10-02: The system shall flag any block scoring ≤ 2 as "stalled," shown with an amber badge and an auto-flag banner. - FR-M10-03: The system shall support Create (not Update or Delete) on Coaching Notes via the generic

sub-item Add — this is a known gap (Section 10.3): a coaching note, once logged, cannot be corrected or removed from this module, and is displayed read-only again in M17. - FR-M10-04: The Desire-diagnosis workflow shall be a bespoke two-step assistive flow, distinct from the generic AI Use Case card pattern: step one diagnoses the stalled Desire score citing Resistance Log and sentiment evidence; on confirmation, step two offers to draft a manager coaching script. Each step requires an explicit Confirm/Discard decision.

Frameworks integrated. ADKAR (Prosci) — the primary individual-level model this module powers directly.

Primary users. Change Manager, People Manager/Coach, Employee (self-report).

6.2.5 M11 — Emotional & Transition Layer

Purpose. Where ADKAR reports what capability or motivation is missing, this layer reports what someone is actually feeling: the Bridges Transition position (Ending → Neutral Zone → New Beginning) and a Kübler-Ross-derived sentiment stage, both under justified-change governance, plus the Divergence Pattern alert that gives journi its “hidden resistance” early-warning signal.

Data captured. bridgesPhase (justified), bridgesNote; sentimentStage (justified, or inferred from sentimentSnapshot free text via keyword match when not explicitly set), sentimentSnapshot note.

Functional requirements. - FR-M11-01: Both bridgesPhase and sentimentStage shall follow the stage-then-justify pattern. - FR-M11-02: When sentimentStage is not explicitly set, the system shall infer one of commitment/exploration/denial/resistance from keyword matching against the free-text sentiment snapshot, defaulting to “exploration” if no keyword matches. - FR-M11-03: The system shall raise a Divergence Alert whenever Knowledge ≥ 3 AND Ability ≥ 3 (strong demonstrated capability, per M10) while the Bridges position still reads “Ending” — a classic hidden-resistance pattern — and shall surface this same flag to M8’s Phase Gate open-flags field.

Frameworks integrated. William Bridges’ Transition Model; Kübler-Ross Change Curve (adapted for organizational, not grief, context).

Primary users. Change Manager, People Manager/Coach.

6.2.6 M12 — Change Risk Register

Purpose. A risk register purpose-built for the people side of change — adoption risk, sponsor-attrition risk, and change-saturation risk from concurrent initiatives targeting the same population — distinct from generic project risk, with a per-risk Mitigation Action Plan tracked to closure.

Data captured. Risk: category (adoption/sponsorship/capacity/saturation), description, likelihood (1–5), impact (1–5), owner, status (justified-change). Mitigation Action (nested per risk): description, owner, dueDate, status.

Functional requirements. - FR-M12-01: The system shall support Create and Delete on risk records; a risk’s core fields (category, description, likelihood, impact, owner) have no in-place Edit once created in the current build — only status is changeable

post-creation, via the justified-change flow. This is a known gap (Section 10.3) for a rebuild targeting full risk-record editability. - FR-M12-02: The system shall support full Create, Update, and Delete on a risk's nested Mitigation Actions. - FR-M12-03: The system shall compute Risk Score as likelihood × impact, flag scores ≥ 12 as high-severity (red) and ≥ 8 as elevated (amber), and sort the risk list by score descending. - FR-M12-04: The system shall surface a saturation banner whenever other active Change Management Projects exist in the same Organization.

Frameworks integrated. Extends standard project-risk practice with change-saturation and adoption-risk categories.

Primary users. Change Manager, PMO, Sponsor.

6.2.7 M13 — Sponsor & Coalition

Purpose. Tracks Executive Sponsor visibility under justified-change governance, the guiding-coalition roster, and a sponsor-action roadmap — the single factor Prosci research identifies as the top predictor of change success, and the substance of Kotter's Step 2.

Data captured. sponsor.visibility (weak/moderate/strong, justified), visibilityNote; Coalition Member: name, role, influence (1-5), engagement (1-5); Sponsor Action: action text, phase, done.

Functional requirements. - FR-M13-01: sponsor.visibility shall follow the stage-then-justify pattern. - FR-M13-02: The system shall support Create (not Update or Delete) on Sponsor Actions, plus a done-toggle. This is a known gap (Section 10.3): once added, a sponsor action's text cannot be edited or removed. - FR-M13-03: The Coalition Member roster has no Add/Edit/Delete UI in the current build — it is read-only display only, populated at seed time. This is a more significant known gap (Section 10.3) given the module's name; a rebuild should treat coalition-roster CRUD as a real requirement even though the reference implementation does not yet expose it. - FR-M13-04: The system shall raise a red alert banner whenever sponsor.visibility == 'weak', cross-referencing M10's stalled Desire scores in the alert text.

Frameworks integrated. Prosci Sponsor Model (active/visible sponsorship); Kotter Step 2 — Build a Guiding Coalition.

Primary users. Change Manager, Sponsor, PMO.

6.2.8 M14 — Communication Planning & Execution

Purpose. Plans and tracks the message-audience-channel-timing matrix driving Awareness and Desire, and operationalizes Kotter's Step 4.

Data captured. message, audience, channel, sender, timing, linked ADKAR block, status (draft/scheduled/sent).

Functional requirements. - FR-M14-01: The system shall support full Create, Read, Update, and Delete on communications, gated by canWrite. - FR-M14-02: The system shall surface a change-saturation banner whenever other active Change Management Projects exist in the same Organization, matching M12's saturation logic. - FR-M14-03:

The communications log shall offer client-side CSV export (Section 6.6.6) to every role, independent of write access.

Frameworks integrated. Kotter Step 4 — Communicate the Vision; feeds ADKAR Awareness/Desire.

Primary users. Change Manager, Communications Practitioner, Sponsor.

6.2.9 M15 — Training & Capability Building

Purpose. Maps directly to ADKAR’s Knowledge and Ability blocks, tracking curriculum coverage, completion, and demonstrated capability — “trained” versus “certified/capable” as two distinct, separately-governed states.

Data captured. curriculum, track, level (Foundation/Practitioner/Advanced), module, objectives/prerequisites/agenda/expectedResults/workshops, targetAudience, facilitator, format, completion (0-100%), certified (justified-change, distinct from completion).

Functional requirements. - FR-M15-01: The system shall support full Create, Read, Update, and Delete on training/curriculum records, gated by `canWrite`. - FR-M15-02: A change to certified shall follow the stage-then-justify pattern, independent of the general record-edit flow, since certification is a governed capability claim, not a data-entry field. - FR-M15-03: The system shall surface a training-needs banner whenever M10 reports Knowledge or Ability ≤ 2 , citing the gap’s source module.

Frameworks integrated. ADKAR Knowledge & Ability blocks; Kotter Step 5 — Enable Action by Removing Barriers.

Primary users. Change Manager, Trainer/Practitioner, People Manager.

6.2.10 M16 — Resistance Management

Purpose. Structured logging and resolution of resistance by type (role/skill/will/systemic), with root cause, severity, and mitigation tracked under justified-change status governance, systemic-pattern detection across cohorts, and a Qualitative Coding Workbench for tagging free-text coaching notes and resistance entries against an organization-scoped codebook.

Data captured. Resistance entry: type, source, rootCause, severity (1-5), mitigation, owner, dueDate, status (justified-change), anonymous flag. Codebook entry (org-scoped): label, description. Code Tag: codeId, sourceType, sourceId, optional linkedResistanceId.

Functional requirements. - FR-M16-01: The system shall support full Create, Read, Update, and Delete on resistance-log entries, gated by `canManage` (= `canWrite`). - FR-M16-02: A change to status (open → in progress → closed) shall follow the stage-then-justify pattern, separate from the general record-edit flow. - FR-M16-03: An Employee role shall be permitted to submit a resistance/concern entry (optionally anonymous) via a “submission only” path (`canSubmit` = `canWrite` OR `role === 'employee'`), without gaining status-management, classification-edit, or delete rights. - FR-M16-04: The system shall raise a red pattern-detection banner when two or more systemic-type entries are logged, framing the pattern as organizational rather than individual and pointing to M13. - FR-M16-05: The Qualitative Coding Workbench shall support Create

and Delete (not Update) on codebook entries — a known gap (Section 10.3) — and Create/Delete on code tags, drawing taggable source material live from this project's own resistance-log entries and M10's coaching notes, with an optional cross-link from a coaching-note tag to a specific resistance-log entry. - FR-M16-06: The Workbench shall compute a tag-frequency rollup relative to the most-used code, surfacing recurring themes a single practitioner reading each note individually would likely miss.

Frameworks integrated. Kübler-Ross resistance/anger stage; Kotter Step 5.

Primary users. Change Manager, People Manager, Employee (submission only).

6.2.11 M17 — Manager-as-Coach Enablement

Purpose. Operationalizes the research finding that a direct manager is the single most influential factor in individual adoption: a team-scoped ADKAR heatmap restricted to a People Manager's own direct reports, a justified Manager Readiness rating, and a read-only view of coaching notes and AI-generated coaching scripts, without exposing organization-wide change data the manager doesn't need.

Data captured. managerReadiness (1-5, justified-change, default 3).

Functional requirements. - FR-M17-01: managerReadiness shall follow the stage-then-justify pattern; the ADKAR heatmap and coaching-note list are always view-only in this module regardless of role, by design — this module deliberately has no add/edit/delete UI of its own for coaching notes, directing users to M10 instead. - FR-M17-02: The heatmap shall be scoped strictly to the manager's own direct reports; it must never expose organization-wide change data through this module.

Frameworks integrated. Prosci Manager/Coach research; ADKAR at the team level.

Primary users. People Manager/Coach.

6.2.12 M18 — Journey Map / Visual Core

Purpose. The signature interface of journi: a literal, visual SVG timeline (the Bridges Ending → Neutral Zone → New Beginning curve) plotting project- or organization-level events with a "Today" marker — deliberately distinct from the tabular dashboards elsewhere, and the visual identity the product takes its name from.

Data captured. Journey Event: label, offsetDays (relative to today), type (milestone/communication/training/assessment).

Functional requirements. - FR-M18-01: The system shall support full Create, Read, Update, and Delete on journey events, gated by canWrite. - FR-M18-02: An Organization-level zoom shall aggregate journey events across every Change Management Project in the Organization onto one chart, in addition to the default single-project view. - FR-M18-03: The "Share Snapshot" control is a demo-only confirmation in the current build with no real export/share backend; a production rebuild wanting an actual shareable artifact must implement this as new functionality, not assume it exists.

Frameworks integrated. Visual synthesis of ADKAR, Bridges, and Kübler-Ross on a single timeline.

Primary users. All roles, at the scope permitted by RBAC.

6.2.13 M19 — Stakeholder Journeys, Touchpoints & Analytics

Purpose. The experience-layer companion to M20's score-centric dashboards: eight persona/exception/sector-specific/system journeys, twenty-four concrete touchpoints across six of those journeys with auditable success criteria, five journey-analytics dashboard definitions (two with genuinely live computed metrics), and a per-project context-overlay reference table.

Data captured (mostly static reference). Journey: id, predecessorId, name, type, trigger, audience, duration, linked modules. Touchpoint: journeyId, PDCA sub-phase, sequence, day offset from trigger, owner role, success criterion, evidence requirement. Touchpoint Log (per project, user-writable): touchpointId, date, note.

Functional requirements. - FR-M19-01: Journeys, Touchpoints, Dashboards, and the Context Overlay are static reference data with no CRUD UI; only the Touchpoint Log is user-writable. - FR-M19-02: The system shall support Create (not Update) and Delete on Touchpoint Log entries, gated by canWrite. - FR-M19-03: The "End User Journey Completion" dashboard shall compute, live, the percentage of the End User Adoption journey's touchpoints with at least one log entry for the current project. - FR-M19-04: The "Sponsor & Charter Compliance" dashboard shall compute, live, the percentage of all M5 Charter Actions with at least one completion log entry for the current project, reading M5's charterActionLog directly. - FR-M19-05: The remaining three dashboards (Mentoring Progression, Divergence Case Resolution, Executive Roll-Up) shall render as descriptive reference cards, since the reference build does not model per-mentee or per-case granularity separately from the underlying M5/M12 logs a BI-connected deployment would aggregate.

Frameworks integrated. Cross-cutting — realizes M18's Journey Map at the persona-definition and touchpoint level of detail; the Divergence Case journey operationalizes M11's Divergence Alert as a recovery workflow; the Mentee journey operationalizes M5's Mentoring Progression model.

Primary users. Change Manager, Training Lead (touchpoint logging); all roles (read).

6.2.14 M20 — Metrics & Analytics Dashboard

Purpose. The analytical brain of journi: the Composite Readiness Index, ADKAR heatmaps, an adoption-curve chart, sentiment/adoption-speed correlation, benchmarking against seeded reference bands, and a cross-transformation-type comparison matrix — rolled up at Project, Organization, or Group level depending on the signed-in role.

Data captured. Purely a computed/aggregated reporting surface — no user-entered fields beyond the roll-up level selector.

Functional requirements. - FR-M20-01: The level selector shall offer Project level to every role; Organization level only to a role scoped Organization-or-broader; Group level only additionally when the scoped Organization actually belongs to a Group and the role is Super Admin or Group Admin — an ungrouped Organization must never show a Group tab, for any role. - FR-M20-02: The system shall compute a benchmarking standing (Behind / In Line / Ahead) per project against a seeded

reference band for its current Lewin phase and against the peer average of every other project in scope at the active level. - FR-M20-03: The Cross-Type Comparison Matrix shall contrast all eight transformation types side by side — typical duration, terminal gate, external-party involvement, dominant framework, and reversibility — each row linked to a seed-project example. - FR-M20-04: The Analytics heatmap and Benchmarking tables shall offer client-side CSV export (Section 6.6.6). - FR-M20-05: This module shall perform no write operations of its own — it is read-only by design, at every roll-up level.

Frameworks integrated. Synthesizes every framework in the platform into one readiness signal, benchmarked against phase-appropriate reference bands.

Primary users. Change Manager, Sponsor, Executive Viewer, PMO.

6.2.15 M21 — Reinforcement & Sustainment

Purpose. The most commonly neglected stage of change work: post-go-live adoption checkpoints, Quick Wins celebration, a lessons-learned (REX) institutionalization log, and the formal sustainment sign-off — Lewin’s Refreeze.

Data captured. Checkpoint (fixed set of three: 30/60/90-day): adoptionRate, regressionRisk, status. Quick Win: title, date. Lesson Learned: text, linkedRuleOrControl (naming which rule/control/charter now encodes it), status (auto-derived). Sign-off: boolean.

Functional requirements. - FR-M21-01: “Record checkpoint” shall be a one-click completion action that generates an adoption-rate value and derives regressionRisk from it (> 80% low, > 60% moderate, else high); the three checkpoints are a fixed set with no add/delete. - FR-M21-02: The system shall support Create (not Update or Delete) on Quick Wins — a known gap (Section 10.3). - FR-M21-03: The system shall support Create and Update (limited to the linkedRuleOrControl field) on Lessons Learned, with no Delete — a known gap (Section 10.3); a lesson’s status shall auto-flip to “applied” the moment a link is entered, “pending” otherwise. - FR-M21-04: Sign-off shall be a simple boolean toggle representing the Lewin Refreeze state, referenced by M8’s Phase Template “Sustain” phase-gate criteria.

Frameworks integrated. Lewin Refreeze; Kotter Steps 6 (Generate Short-Term Wins) and 8 (Anchor New Approaches).

Primary users. Change Manager, People Manager, Sponsor.

6.2.16 M22 — Field Notes

Purpose. A lightweight freeform log for change-management knowledge that doesn’t fit a structured module — workshops, decisions, sign-offs, nominations, handoffs — explicitly positioned as a scratchpad, not a substitute for a structured module: once something becomes a real record elsewhere, it belongs there instead.

Data captured. date, category (Workshop/Decision/Sign-Off/Nomination/Handoff/Other), relatedModules (multi-select across all M1-M22), title, body, author.

Functional requirements. - FR-M22-01: The system shall support full Create, Read, Update, and Delete on field notes, gated by canWrite. - FR-M22-02: relatedModules

shall be a free multi-select tag against the current M1-M22 module list — a descriptive cross-reference only, not a structural foreign key into another module's data. - FR-M22-03: The list shall be filterable by category, with each category color-coded (Workshop = brand, Decision/Sign-Off = green, Nomination = amber, Handoff/Other = gray).

Frameworks integrated. None directly — a cross-cutting practitioner utility.

Primary users. All roles with project write access.

7. Cross-Cutting Functional Requirements

These requirements are not owned by any single module — they are shared infrastructure that most or all of M1-M22 draw on identically.

7.1 CSV Export

- FR-X-01: journi's highest-traffic tables shall offer a client-side CSV export — Sponsor Coalition (M13), Communications (M14), Training & Certification (M15), Risk Register (M12), and the Analytics readiness heatmap and benchmarking tables (M20) — built entirely in the browser via a Blob download, with no backend export service required.
- FR-X-02: Each export shall reflect exactly the rows and columns currently on screen, respecting the same RBAC and scope filtering as the table itself.
- FR-X-03: Every export shall be written with a UTF-8 byte-order mark, so accented French and Arabic content opens correctly in Excel.

7.2 Notification Center (Alerts)

- FR-X-04: A bell icon in the TopBar shall surface the platform's defined alert conditions as a persistent, dismissible, per-project in-app log.
- FR-X-05: The following conditions shall fire live, computed client-side from data journi already holds, whenever true for the scoped Change Management Project: a Divergence Pattern (M11); a Critical post-go-live regression-risk score (M21); a Sponsor Coverage Gap — visibility rated Weak (M13); a Resistance Escalation Threshold breach — three or more open resistance-log entries (M16); a Change Saturation breach — two or more other concurrent initiatives targeting the same Organization (M12/M14); a Communication Overload — more than three not-yet-sent communications scheduled across a project and its concurrent same-Organization initiatives (M14); a Phase Gate closed No-Go or Go-with-Conditions (M8); a Guiding Coalition Gap — fewer than two named coalition members (M13); and a blocked Sustainment sign-off (M21).
- FR-X-06: Dismissing an alert shall persist per Change Management Project (in `dismissedAlerts[]`, Section 4.3) and survive a page reload; a Restore control shall bring a dismissed alert back into view.
- FR-X-07: The alerts themselves shall never be persisted — only the dismissal state is; every alert is recomputed live on each render from current project data.
- FR-X-08: The following conditions are catalogued for traceability but depend on backend/data-model infrastructure the reference build does not have, and never fire: survey-exception retries, AI-confidence scoring, import integrity,

administrative account lock-out, GDPR request SLA, AI provider fallback, and Champion Coverage Below Target (the last excluded because journi has no structured champion-tracking data model, not a backend gap). A rebuild adding real outbound delivery (email/push/Teams) for the live-firing conditions above should treat that as new infrastructure, not an existing capability being exposed.

7.3 Justification Governance

- FR-X-09: Every score or state change to a Change Management Project’s ADKAR block (M10), Lewin macro-state (M7), Bridges/Kübler-Ross position (M11), sponsor visibility (M13), manager readiness (M17), training certification (M15), resistance status (M16), or risk status (M12) shall follow the same stage-then-justify pattern: the practitioner stages the new value, writes a justification note, then saves — at which point the new value and its justification are appended together, in one atomic update, to that project’s Change Log (M7).
- FR-X-10: A platform-wide requireJustification toggle (M2, Governance Settings, default on) shall determine whether the justification note is a hard requirement before Save enables, or merely offered and recorded when given.
- FR-X-11: Justification text is expected to cite evidence already observed, never a plan for future evidence; journi does not enforce this discipline programmatically, by design — it is a record of practitioner judgment, and the tool’s role is to make that judgment easy to capture and impossible to lose, not to police its content.

7.4 Permission Matrix

See Section 3.7 and FR-M2-02 through FR-M2-03 for the full specification; in summary, the role × capability grid on M2 is the single runtime override every can* check in Sections 3.7 and 6 reads from, editable in place by a Super Admin, taking effect immediately and platform-wide without a code change. Section 10.3 notes the one known module (M8) that does not currently read from it.

7.5 AI Governance Tiers

See Section 6.1.6 (M6) FR-M6-01 through FR-M6-10 for the full specification of the Assistive/Augmented tier model and its activation, override, and audit-logging requirements.

7.6 Framework-to-Module Mapping

journi does not force a single change methodology; each established framework is mapped onto the module(s) where it is operationally expressed, so practitioners trained in any one methodology find their vocabulary represented in the tool. Module numbers below use the current M1–M22 sequence (Section 1.6).

Framework	Primary Level	Modules Where Applied
ADKAR (Prosci)	Individual / Cohort	M10 ADKAR Engine; M15 Training; M17 Manager as Coach; M20 Analytics
Kotter’s 8 Steps	Organizational	M13 Sponsor & Coalition; M14 Communications; M15 Training; M21 Sustainment

Framework	Primary Level	Modules Where Applied
Lewin (Unfreeze-Change-Refreeze)	Organizational (macro-state)	M7 Initiative Registry; M21 Sustainment
Bridges Transition Model	Individual / Emotional	M11 Emotional & Transition; M18 Journey Map
Kübler-Ross Change Curve	Individual / Emotional	M11 Emotional & Transition; M16 Resistance Management
Prosci Sponsor & Manager Research	Organizational / Team	M13 Sponsor & Coalition; M17 Manager-as-Coach
AI Tiers — Assistive & Augmented	Cross-cutting	M6 AI Use Case Library; expressed inside M9, M10, M11, M13, M14, M15, M16, M17, M18, M20, M21

M8 (WBS & Gantt) is deliberately cross-cutting rather than tied to one row above: it gives every framework in this table a single baseline-vs-actual calendar alongside the Project Management delivery track, next to the Lewin macro-state owned by M7, the Bridges/Kübler-Ross position owned by M11, and the ADKAR scores owned by M10. M5 (Charter Registry) and M19 (Journeys, Touchpoints & Analytics) are cross-cutting in the same sense — rather than introducing frameworks of their own, they operationalize the Prosci and Kotter rows above as signed, trackable behavioral standards (M5) and give the Bridges/Kübler-Ross journey M18 already visualizes a touchpoint-level completion record (M19).

8. Non-Functional Requirements

8.1 Performance

- NFR-01: The frontend build shall load and render the largest single-project view within normal broadband/LAN latency without a loading spinner exceeding a few seconds — the entire application ships as one JS bundle (currently ~660KB minified, ~180KB gzipped) with no code-splitting; a rebuild adding substantially more module content should consider manualChunks splitting if the bundle grows enough to trigger Vite’s default 500KB warning meaningfully.
- NFR-02: State-save shall be debounced 400ms after the last change (Section 3.4) to avoid saturating the backend with a write per keystroke, at the deliberate cost of up to 400ms of unsaved-edit exposure if the process is killed mid-edit.

8.2 Security

- NFR-03: **The reference build has no authentication at the HTTP layer.** GET/PUT /api/state accept any caller with no login, session, token, or credential check whatsoever — RBAC (Section 3.7) is enforced entirely client-side, in the React app, as a UX/workflow control, not a security boundary. A user with direct HTTP access to the server (or its port) can read or overwrite the *entire* application’s data for every tenant, regardless of role. This is acceptable for the reference build’s intended context — a single machine on a trusted LAN or a single user’s desktop — but is not a production security posture. A production or multi-tenant-hosted rebuild must add real server-side

authentication and per-request authorization before RBAC can be considered anything more than a workflow convenience.

- NFR-04: The reference build implements no TLS termination, rate limiting, or web-application-firewall behavior; these must be added at the infrastructure layer for any internet-facing deployment.
- NFR-05: The API performs no request-body shape validation on `PUT /api/state` — a malformed body will overwrite the stored state with whatever was sent. A production rebuild should validate the body against the Section 4 schema before persisting it.
- NFR-06: An AI use case shall never grant a user visibility they would not otherwise have (FR-M6-10); the Real LLM Provider Connection's API key shall never be persisted server-side or included in any data export (FR-M6-08).
- NFR-07: Individual-level ADKAR scores and sentiment data shall be restricted, by RBAC default, to roles in `ROLES_WITH_INDIVIDUAL_VISIBILITY` (Super Admin, Group Admin, Organization Admin, Change Manager, People Manager); Sponsors and Executives see aggregated, de-identified views only, per M20's roll-up logic.

8.3 Reliability & Data Integrity

- NFR-08: The reference build's persistence layer is a single SQLite file with no replication and no automated backup; `journi.db` is the entire durability boundary. A production deployment should add scheduled backups with a defined retention policy at minimum, and should evaluate whether the single-writer-row model (Section 3.3) meets its concurrency needs before scaling beyond one small team's usage.
- NFR-09: Every write is a full-document overwrite with no optimistic-locking or versioning at the transport level (Section 3.4) — two browser tabs editing the same tenant concurrently will silently clobber each other's changes on the losing tab's next debounced save. A rebuild targeting concurrent multi-user editing of the same project must add conflict detection or move to per-entity writes (Section 5.2).
- NFR-10: `better-sqlite3` is deliberately kept as an `optionalDependency` (Section 3.3) specifically so a failed native build never aborts `npm install` as a whole, given `node:sqlite` is tried first and covers the common case.

8.4 Usability & Accessibility

- NFR-11: Every layout (navigation, tables, charts, the Journey Map) shall render correctly mirrored under Arabic RTL, not just with mirrored text direction (Section 3.6).
- NFR-12: Status, severity, and classification fields shall use the shared Badge component's consistent color vocabulary across all 22 modules (Section 5.1), so a user who has learned the color meaning in one module recognizes it in every other.
- NFR-13: M17's interface shall remain lightweight enough for a time-constrained frontline supervisor to use on a mobile browser, per its stated design intent, even though no dedicated mobile layout exists elsewhere in the application.

8.5 Localization

See Section 3.6 for the full specification. In summary: English, French, and Arabic (with full RTL) are first-class, not translated-as-afterthought; every UI string lives in one dictionary file; language precedence resolves user preference → Organization default → English fallback on every sign-in and Organization switch.

8.6 Portability / Installability

- NFR-14: `npm install` must succeed on a fresh Windows machine with only Node.js installed — no Visual Studio Build Tools, no Python, no C++ toolchain requirement — across the range of Node 18 through the latest release, satisfied by the dual SQLite driver strategy (Section 3.3).
- NFR-15: The same source must run unmodified on macOS/Linux via standard `npm install` & `npm run build / npm start`, without requiring the Windows batch-file wrappers.

8.7 Maintainability / Extensibility

- NFR-16: Adding a new module shall follow the established pattern: a new `Module<N>Page.jsx` (internal identifier, Section 1.6), a route entry, a Sidebar entry with a `navM<N>` translation key (whose *value*, not key, carries the visible module number), `i18n` keys for every user-facing string in all three languages, and — if it needs its own persisted sub-collection — a new array on the `cmProject` object (Section 4.3) plus `addSubItem/updateSubItem/removeSubItem` wiring rather than a bespoke mutator, unless the module's data genuinely doesn't fit the generic sub-item shape (as with M2's Permission Matrix or M7's Change Log).
- NFR-17: A module's visible number and its internal file/route identifier are permanently decoupled (Section 1.6) specifically so a future renumbering never requires a mechanical rename sweep across the codebase — only the translation dictionary's *values* change.

8.8 Compatibility

- NFR-18: The application targets evergreen desktop browsers (Chrome, Edge, Firefox); no legacy-browser support (e.g. Internet Explorer) is in scope.
-

9. Seed / Reference Data Requirements

9.1 Reference Tenant

The reference build seeds one Group (“Atlas Industrial Group”) containing four Organizations, each carrying its own Main Project(s) and linked Change Management Project(s), for a total of 20 seeded Change Management Projects — one per seeded `change_manager`-role user. Fourteen of these are illustrative “cases” spanning three sectors (Manufacturing, Logistics & Transportation, Health) crossed with eight transformation types, documented in full narrative detail in the predecessor Functional Specification (Section 6) and in the Master User Guide. The remaining seeded projects support the guide's own running scenario tenant. All seeded

organizations, individuals, and figures are fictional composites; they are not a substitute for real organizational data before go-live.

Sector	Transformation Types Covered
Manufacturing	ERP Implementation; Business Process Automation; Integrated Management System (QMS); Cultural / Values Transformation; Training & Skills Development
Logistics & Transportation	ERP Implementation; Business Process Automation; Integrated Management System (QMS); Business Process Reengineering; Operating Model Redesign
Health	ERP Implementation; Business Process Automation; Integrated Management System (QMS); Compliance-Driven Change

9.2 Demo User / Role Matrix

The seed data includes 24 platform users spanning all nine roles (Section 3.7): one Super Admin, one Group Admin, one Organization Admin per seeded Organization, one Change Manager per seeded Change Management Project, one People Manager, one Sponsor, one Executive, and one Employee — each with a fixed scopeType/scopeId demonstrating every scope level the RBAC model supports (Section 3.7).

9.3 Shared Catalogs

Seeded once, shared across every tenant rather than duplicated per-Organization: the AI Use Case catalog (14 entries, M6), the Macro Process catalog (10 entries, M4), the E2E Process Registry (17 entries, M4), the Phase Template library (8 entries, M8), the RACSI grid default, the Charter Registry's 8 charter definitions (M5), and the default Permission Matrix (DEFAULT_ROLE_PERMISSIONS, M2).

10. Appendices

10.1 Glossary

See Section 1.4 for framework and access-control acronyms. Module-specific vocabulary is defined inline in each module's Section 6 entry on first use.

10.2 Requirement Traceability

Section 1.6's module table is the authoritative map from a displayed module number to its source file; Section 4 is the authoritative field-level schema; Section 6 is the authoritative functional-requirement list, one FR-Mn-*nn* per testable behavior. A QA team writing test cases against this document should trace each test to one FR id; a dev team scoping a rebuild task should trace each task to one module's FR block plus its Section 4 data-model entry.

10.3 Known Functional Gaps (Reference Implementation)

These are honest, verified-against-source gaps in the current reference build, not aspirational shortcomings — each is called out at its owning module in Section 6 and collected here for rebuild-planning convenience. None blocks the application’s core workflows; each represents either a deliberate scope decision (e.g. an immutable Phase Gate record) or an incompletely-extended CRUD pattern worth completing in a rebuild.

#	Module	Gap	Type
1	M10 ADKAR Engine	Coaching Notes are Add-only — no Edit or Delete, in this module or in M17 where they are also displayed.	Incomplete CRUD
2	M13 Sponsor & Coalition	Coalition Member roster has no Add/Edit/Delete UI at all — read-only display, populated only at seed time.	Missing CRUD
3	M13 Sponsor & Coalition	Sponsor Actions support Add and a done-toggle only — no Edit or Delete of the action text.	Incomplete CRUD
4	M12 Risk Register	A risk’s core fields (category, description, likelihood, impact, owner) have no Edit once created — only status is changeable, via the justified-change flow.	Incomplete CRUD
5	M21 Sustainment	Quick Wins are Add-only — no Edit or Delete.	Incomplete CRUD
6	M21 Sustainment	Lessons Learned support Add and a partial Edit (LinkedRuleOrControl only) — no Delete.	Incomplete CRUD
7	M16 Resistance Management	Codebook entries support Add and Delete only — no Edit of an existing code’s label/description.	Incomplete CRUD
8	M8 WBS & Gantt	Phase Gates support Add and Delete only — no Edit of a recorded joint decision (may be intentional: a gate record is arguably meant to be immutable once logged).	Design decision or gap — confirm intent before rebuilding

#	Module	Gap	Type
9	M8 WBS & Gantt	Write-access checks call <code>canWrite(currentUser?.role)</code> without passing the Permission Matrix, unlike every other module — Super Admin edits to the matrix do not affect this module's write gating.	Inconsistency
10	M18 Journey Map	"Share Snapshot" is a demo-only confirmation message with no real export/share backend.	Stub, not a regression
11	Cross-module	Some in-code comments and the AI Use Case catalog's <code>moduleLabel</code> strings still reference the pre-renumbering module numbers in places; the Section 1.6 table (sourced from <code>Sidebar.jsx/translations.js</code> , not from code comments) is the authoritative current mapping and should be used for any further documentation work.	Documentation drift

10.4 Default Permission Matrix

The seeded default state of `data.rolePermissions` (Section 4.1), editable at runtime on M2:

Role	manage Hierarchy	manage Users	write	activate AiForOrg	request ProjectAiOverride	manage Charers	manage AiUseCases	manage Templates
super_admin	✓	✓	✓	✓	✓	✓	✓	✓
group_admin	✓	✓	✓	✓	✓	✓	✓	✓
org_admin	✓	✓	✓	✓	✓	✓	✓	✓
change_manager			✓		✓	✓	✓	✓
people_manager			✓					
practitioner			✓					
sponsor								
employee								

Role	manage Hierarc hy	manage Users	write	activate AiForOr g	request ProjectA iOverrid e	manage Charter s	manage AiUseCa ses	manage Templat es
executive								

10.5 Closing Notes

The module set and hierarchy described in this document are designed to scale from a single standalone Change Management Project run by one consultant, up to a multi-Organization Group portfolio managed by a central Center of Excellence, without changing the underlying data model. A rebuild team's practical next steps, in order: (1) stand up the two-endpoint whole-state API and dual-driver SQLite persistence exactly as specified in Sections 3–5, since every module's frontend logic assumes that shape; (2) rebuild the 22 module pages against Section 6's functional requirements, reusing the shared component/CRUD conventions in Section 3.2 rather than inventing per-module patterns; (3) close the gaps in Section 10.3 opportunistically as each owning module is rebuilt, rather than as a separate pass; (4) treat Section 8.2's authentication gap as the first production-hardening item before any internet-facing deployment.